Decision Trees for Regression

Neighborhoods via Recursive Partitioning

Objectives

In this note, we will discuss:

- **decision trees** for regression,
- how trees differ from KNN when determining **similarity** of data,
- how to find and evaluate decision tree **splits** for regression,
- and how to **fit** and **tune** decision trees with `DecisionTreeRegressor`.

We'll return to the Auto MPG example to explore how trees split data and make predictions.

Python Setup

```
# basics
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

# machine learning
from sklearn.datasets import make_friedman1, make_regression
from sklearn.tree import DecisionTreeRegressor, plot_tree, export_text
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import root_mean_squared_error, mean_squared_error, r2_sc
from sklearn.model_selection import train_test_split
```

```
# data
from ucimlrepo import fetch_ucirepo
```

Notebook

The following Jupyter notebook contains some starter code that may be useful for following along with this note.

- Regression Trees Notebook

Introduction

Decision trees are a fundamental machine learning algorithm that can be used for both classification and regression tasks. In regression problems, decision trees predict numeric target variables by recursively partitioning the feature space into regions ("neighborhoods") and predicting the mean (or other statistics like the median) of the target values within each region.

Loss Functions

$$\text{MSE}(y, \hat{y}) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y})^2$$

$$\text{MAE}(y, \hat{y}) = \frac{1}{n}\sum_{i=1}^{n}|(y_i - \hat{y})|$$

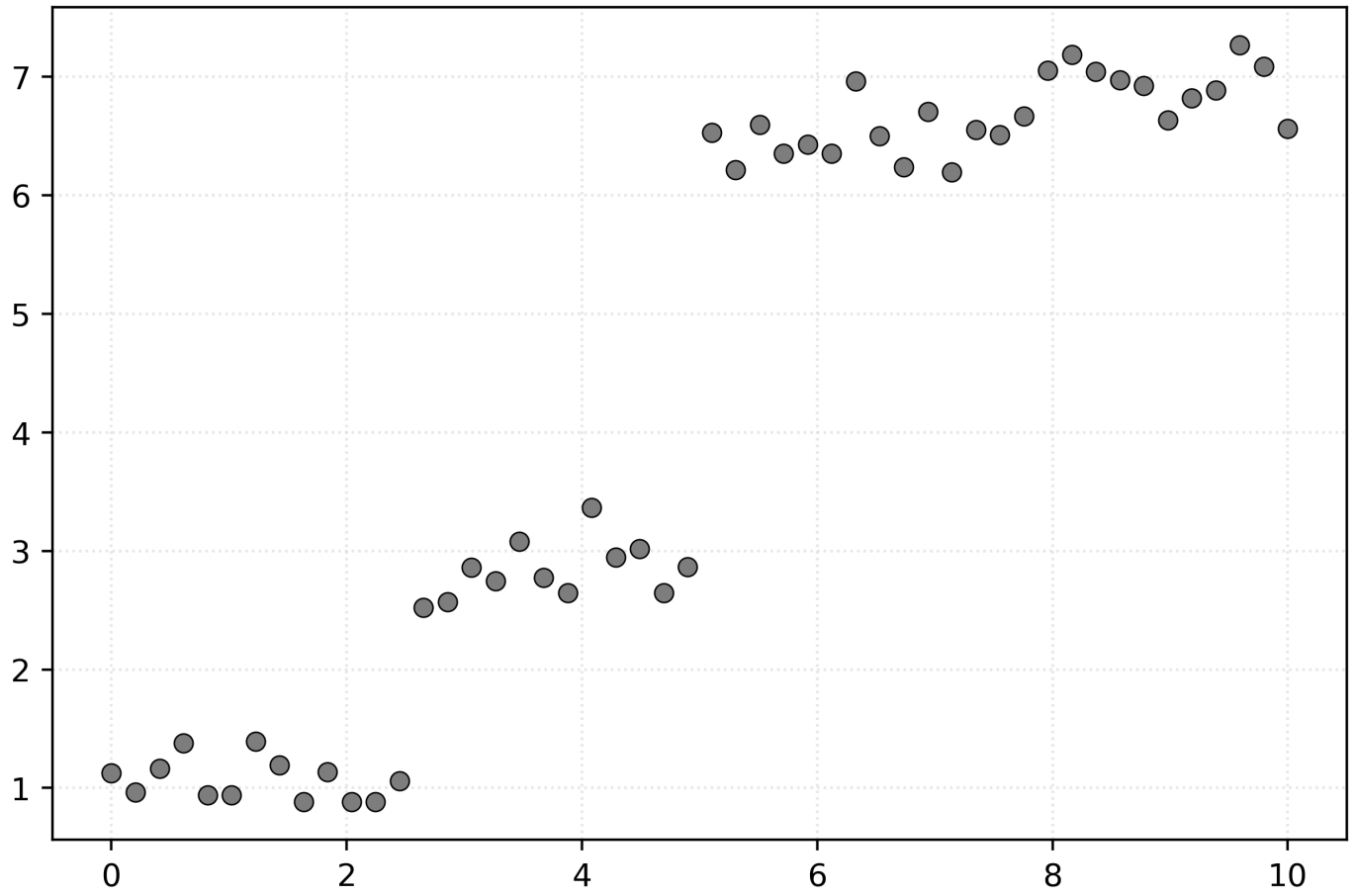
$$\text{SSE}(y, \hat{y}) = \sum_{i=1}^{n}(y_i - \hat{y})^2$$

Tree Building Idea

x	y
2.65	2.52
7.96	7.05
6.12	6.35

x	y
9.18	6.82
3.47	3.08
9.80	7.09
5.31	6.21
5.10	6.53
6.53	6.50
3.88	2.65

Table 1: Ten samples of simulated data with a single feature x and a target y.



Decision trees **split** the feature data into regions (neighborhoods), called **nodes**, by finding optimal **cutoffs**.

> Where KNN looks for neighbors *when making predictions*, decision trees create neighborhoods *during fitting*.

For a single feature x and node N, we choose a cutoff value c_N that divides our data into two groups:

- **Left Node**: $N_L = \{i : x_i \leq c_N\}$
- **Right Node**: $N_R = \{i : x_i > c_N\}$

Each neighborhood predicts the mean of the target values within that region.

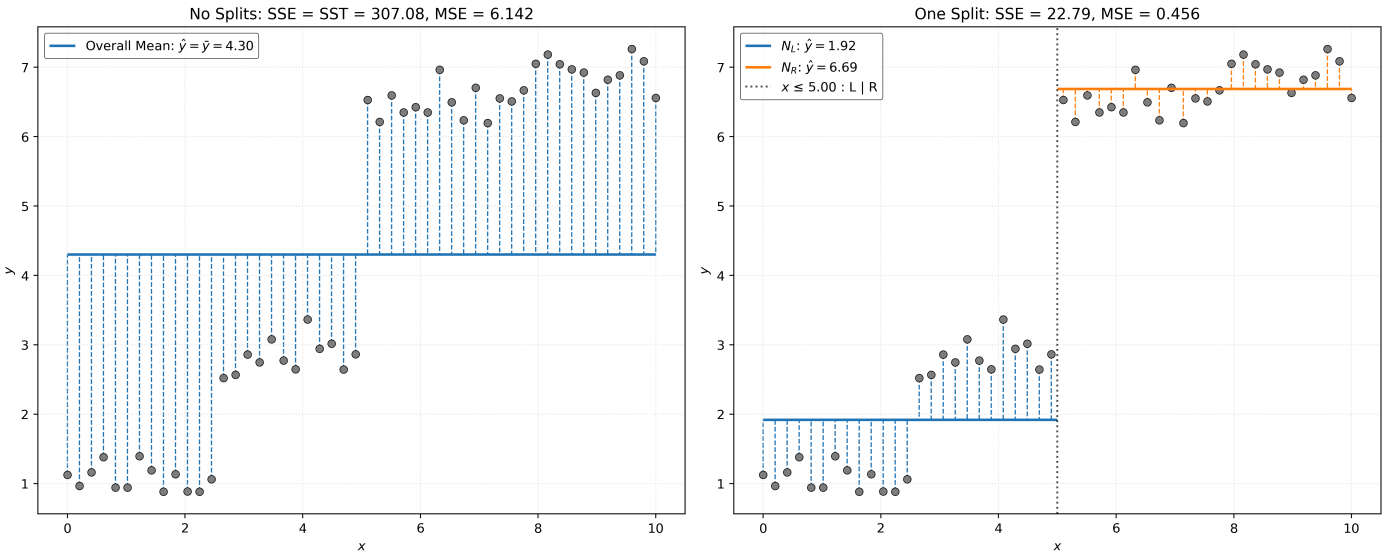


Figure 1: Comparison of SST vs SSE. Left panel shows SST (total sum of squares) with all residuals measured from the overall mean. Right panel shows SSE (sum of squared errors) after one split at x=5, with residuals measured from neighborhood means. The split reduces error by creating more homogeneous regions.

To find the optimal split, we consider all possible features (here just a single feature x) and all possible cutoff values, then choose the combination that **minimizes** SSE.

After splitting node N based on $x_j \leq c_N$, the new SSE is:

$$\mathrm{SSE}(y, \hat{y}) = \sum_{i=1}^{n} (y_i - \hat{y})^2 = \sum_{i \in N_L} (y_i - \bar{y}_L)^2 + \sum_{i \in N_R} (y_i - \bar{y}_R)^2$$

where $\bar{y}_L$ and $\bar{y}_R$ are the means of the left and right nodes, respectively. That is, for samples in the left node, N_L, the predicted value $\hat{y}$ is $\bar{y}_L$. For samples in the right node, N_R, the predicted value $\hat{y}$ is $\bar{y}_R$.

The algorithm considers all features and typically uses **midpoints** between adjacent sorted values as potential cutoffs.

A tree will consist of more than a single split. To develop a "deeper" tree, we **recursively partition** the feature space. That is, after splitting the root node N_0 into nodes N_L and N_R, we repeat the same

process within both nodes.

We first split N_L into N_{LL} and N_{LR}.

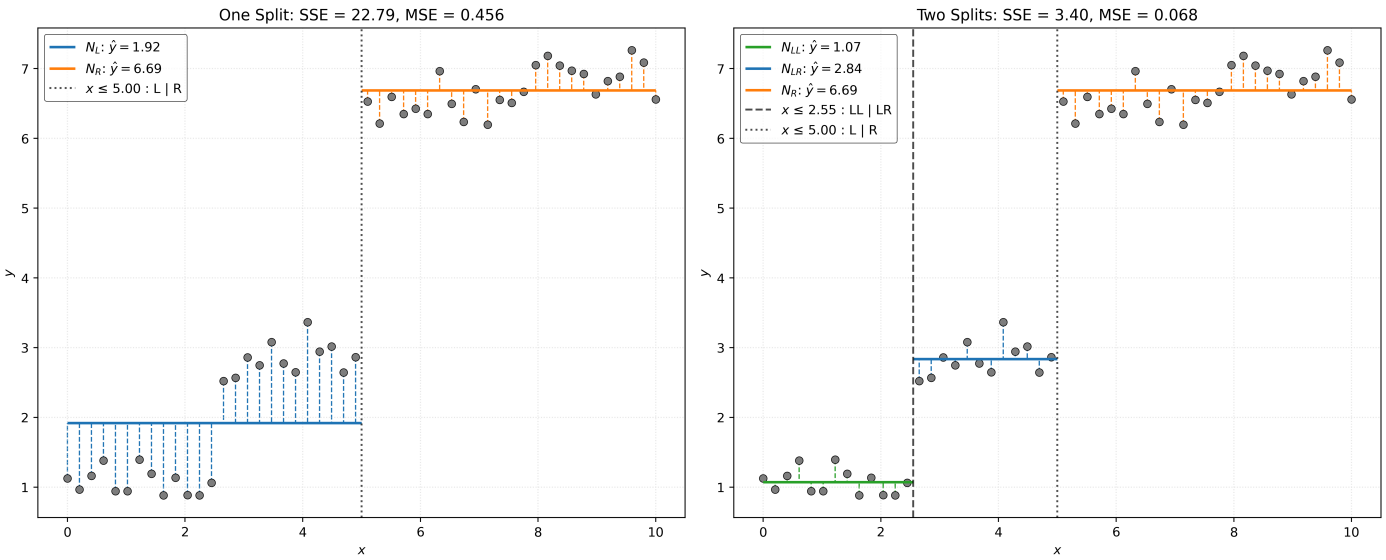


Figure 2: Progressive reduction of SSE through additional splits. Left panel shows one split at x=5 creating two neighborhoods. Right panel shows two splits (at x=2.5 and x=5) creating three neighborhoods. Each additional split reduces SSE by creating smaller, more homogeneous regions with shorter residuals.

With an arbitrary number of neighborhoods T (terminal nodes), we have:

$$\text{SSE}(y, \hat{y}) = \sum_{t=1}^{T} \sum_{i \in N_t} (y_i - \bar{y}_t)^2$$

We then split N_R into N_{RL} and N_{RR}.

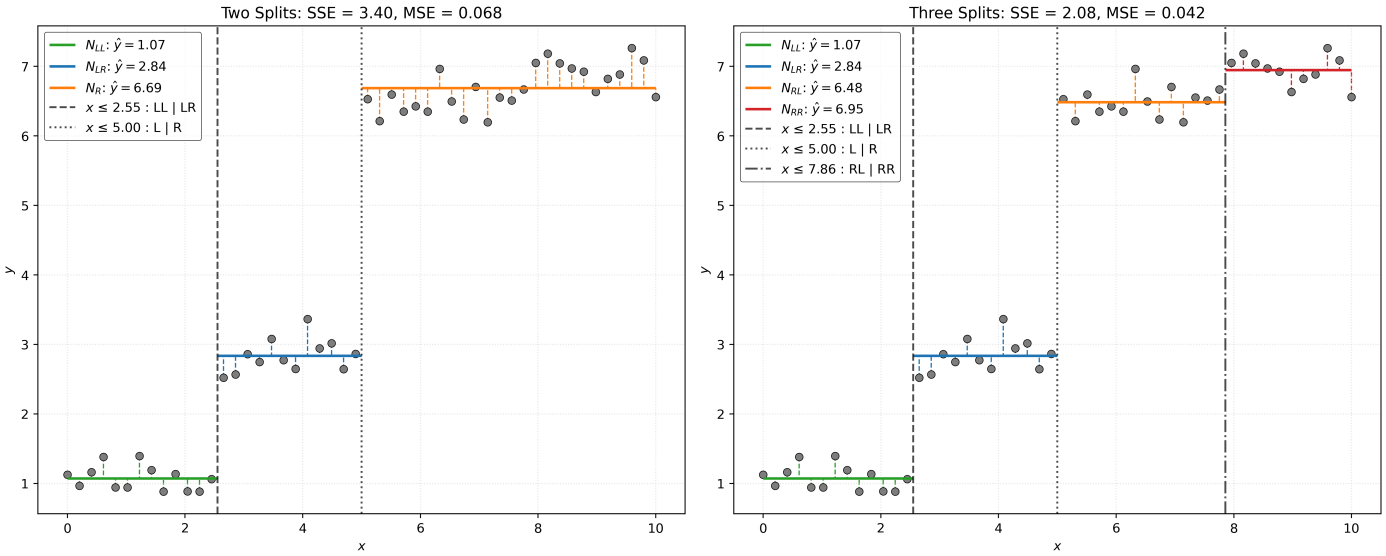


Figure 3: Adding a third split further reduces SSE. Left panel shows two splits creating three regions. Right panel shows three optimal splits creating four regions, each corresponding to our original step function levels.

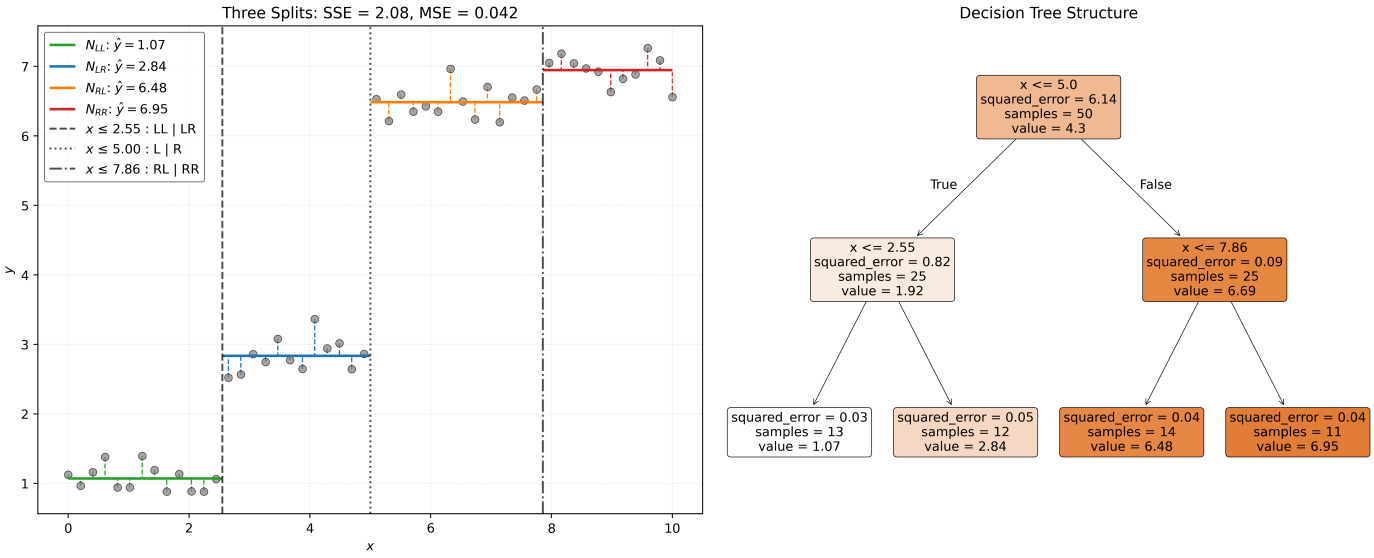


Figure 4: Visualization of decision tree structure. Left panel shows the data with three splits creating four regions. Right panel shows the corresponding tree structure with decision nodes (splits) and leaf nodes (predictions).

With a tree build, we can give categorize the different types of nodes that we have encountered.

- **Root Node**: The "top" node containing all data.
- **Terminal Nodes**: Nodes at the "bottom" of the tree that are not split any further. In the tree analogy, this could also be called **leaves**.
- **Internal Nodes**: Any node that is not the root node or a terminal node.

Decision Tree Tuning Parameters

```
def simulate_sin_data(n, sd, seed):
    np.random.seed(seed)
    X = np.random.uniform(
        low=-2 * np.pi,
        high=2 * np.pi,
        size=(n, 1),
    )
    signal = np.sin(X).ravel()
    noise = np.random.normal(
        loc=0,
        scale=sd,
        size=n,
    )
    y = signal + noise
    return X, y
```

```
X, y = simulate_sin_data(n=200, sd=0.25, seed=42)
```

▶ Show Code for Plot

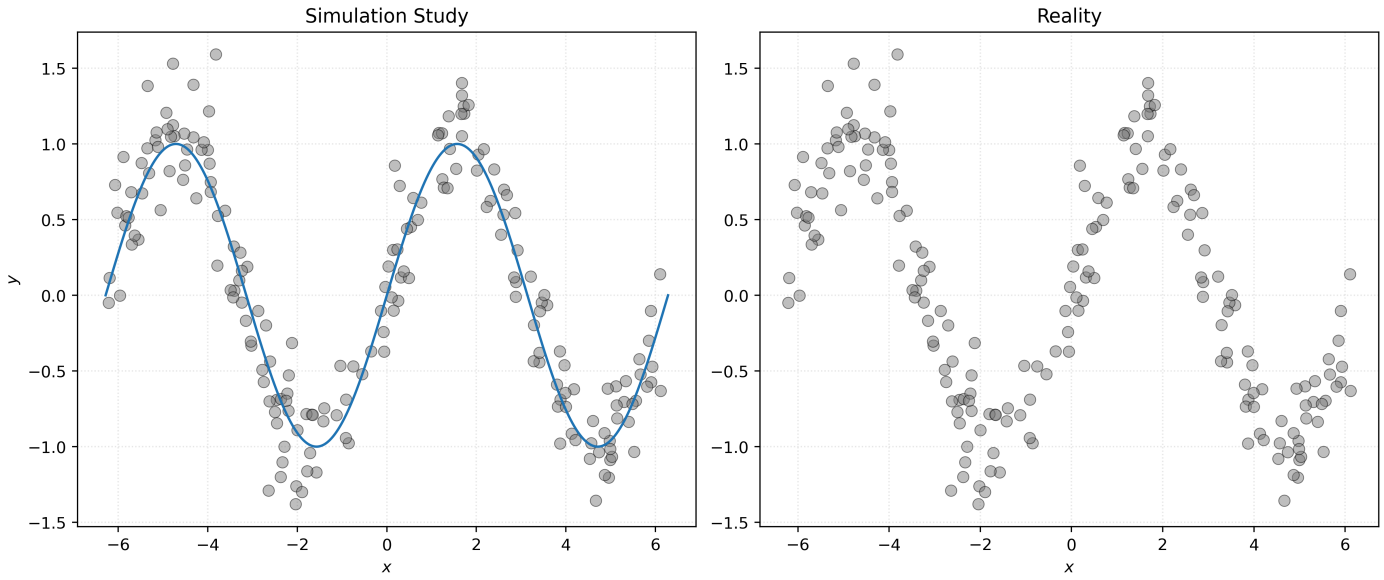


Figure 5: A visual representation of a data generating process, and data generated by the process. **Left**: A sine wave that is the signal and data generated by the process. **Right**: Data generated by the process, without any indication of the true signal.

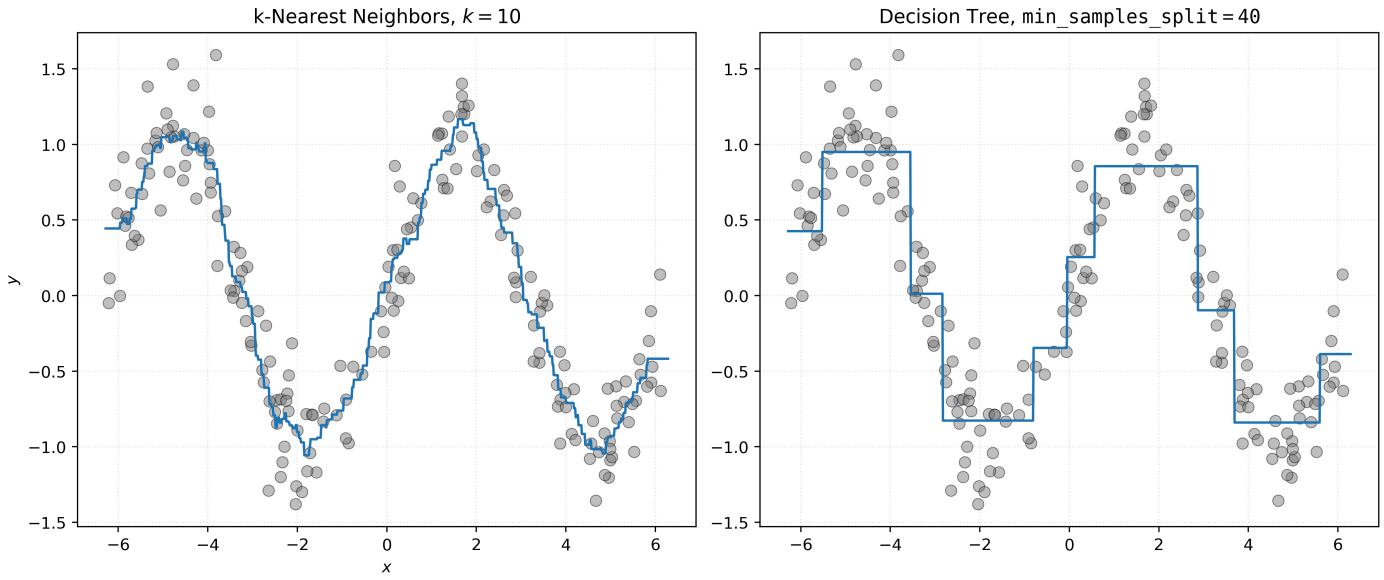
```
# fit a knn model for comparison
knn010 = KNeighborsRegressor(n_neighbors=10)
_ = knn010.fit(X, y)
```

```
# list the possible parameters (and their default values) to the DecisionTreeR
DecisionTreeRegressor().get_params()
```

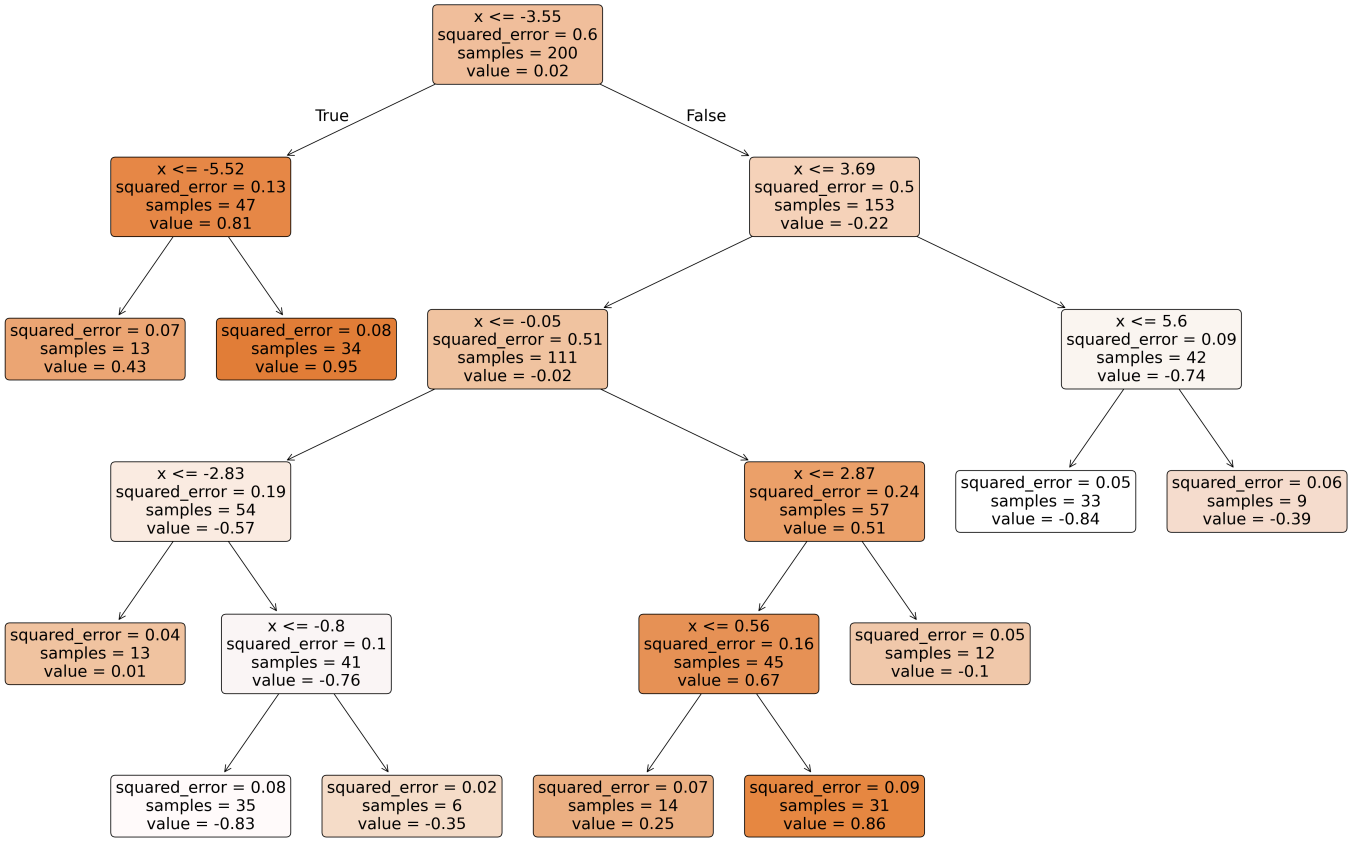
```
{'ccp_alpha': 0.0,
 'criterion': 'squared_error',
 'max_depth': None,
 'max_features': None,
 'max_leaf_nodes': None,
 'min_impurity_decrease': 0.0,
 'min_samples_leaf': 1,
 'min_samples_split': 2,
 'min_weight_fraction_leaf': 0.0,
 'monotonic_cst': None,
 'random_state': None,
 'splitter': 'best'}
```

```
# fit a decision tree with min_samples_split=40
dt040 = DecisionTreeRegressor(min_samples_split=40)
_ = dt040.fit(X, y)
```

▶ Show Code for Plot




```
# visualize the decision tree
fig, ax = plt.subplots(1, 1, figsize=(12, 8))
plot_tree(dt040, feature_names=["x"], filled=True, rounded=True, fontsize=10,
plt.show()
```




```
# view text representation of the tree
print(export_text(dt040))
```

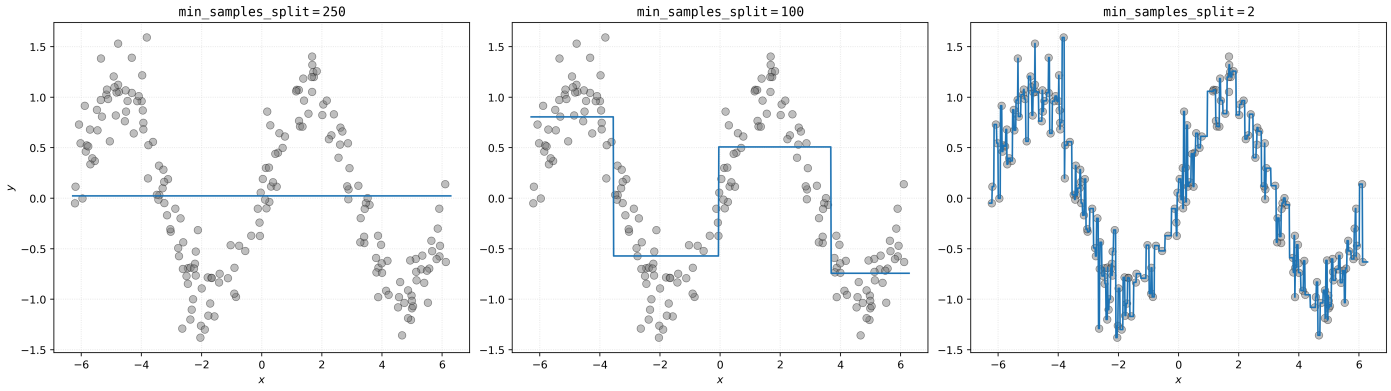
```
|--- feature_0 <= -3.55
|   |--- feature_0 <= -5.52
|   |   |--- value: [0.43]
|   |--- feature_0 >  -5.52
|   |   |--- value: [0.95]
|--- feature_0 >  -3.55
|   |--- feature_0 <= 3.69
|   |   |--- feature_0 <= -0.05
|   |   |   |--- feature_0 <= -2.83
|   |   |   |   |--- value: [0.01]
|   |   |   |--- feature_0 >  -2.83
|   |   |   |   |--- feature_0 <= -0.80
|   |   |   |   |   |--- value: [-0.83]
|   |   |   |   |--- feature_0 >  -0.80
|   |   |   |   |   |--- value: [-0.35]
|   |   |--- feature_0 >  -0.05
|   |   |   |--- feature_0 <= 2.87
|   |   |   |   |--- feature_0 <= 0.56
```

```
|   |   |   |   |   |--- value: [0.25]
|   |   |   |   |--- feature_0 >  0.56
|   |   |   |   |   |--- value: [0.86]
|   |   |   |--- feature_0 >  2.87
|   |   |   |   |--- value: [-0.10]
|   |--- feature_0 >  3.69
|   |   |--- feature_0 <= 5.60
|   |   |   |--- value: [-0.84]
|   |   |--- feature_0 >  5.60
|   |   |   |--- value: [-0.39]
```

```
# initialize decision trees with different values of the tuning parameter min_
dt002 = DecisionTreeRegressor(min_samples_split=2)
dt100 = DecisionTreeRegressor(min_samples_split=100)
dt250 = DecisionTreeRegressor(min_samples_split=250)
```

```
# fit those models
_ = dt002.fit(X, y)
_ = dt100.fit(X, y)
_ = dt250.fit(X, y)
```

▶ Show Code for Plot

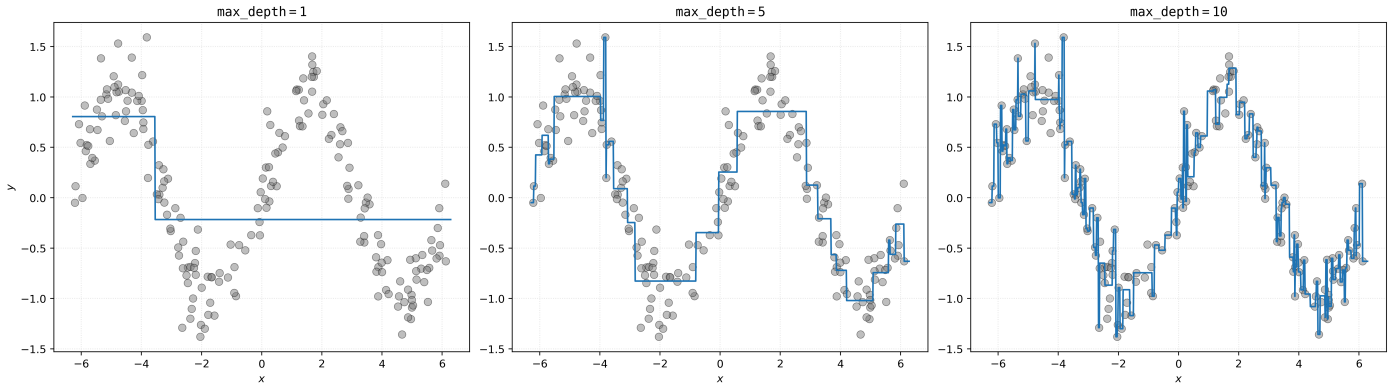



```
# initialize decision trees with different values of the tuning parameter max_
dt_d01 = DecisionTreeRegressor(max_depth=1)
dt_d05 = DecisionTreeRegressor(max_depth=5)
dt_d10 = DecisionTreeRegressor(max_depth=10)
```

```
# fit those models
_ = dt_d01.fit(X, y)
```

```
_ = dt_d05.fit(X, y)
_ = dt_d10.fit(X, y)
```

▶ Show Code for Plot



Recursive Partitioning Algorithm

The decision tree algorithm works through **recursive partitioning**. Let's look at a sketch of the algorithm.

- **Initialize** root node containing all data.
- **Set** stop conditions via `min_samples_split` or `max_depth`.
- **While** there are terminal nodes that can be split:
  - **For** each terminal node N:
    - **If** stop conditions are met:
      - **Skip** node.
    - **For** each feature x_j:
      - **Find** cutoff c_N for potential split $x_j \leq c_N$ that minimizes loss (SSE).
    - **Pick** feature x_j (and associate cutoff c_N) with smallest loss.
    - **Split** N into N_L and N_R using $x_j \leq c_N$, creating two new terminal nodes.

The algorithm considers all features and finds potential split points by examining the data. For continuous features, typical split points are the **midpoints** between adjacent unique values when the data is sorted.

▶ Show Code for Recursive Partitioning Demo

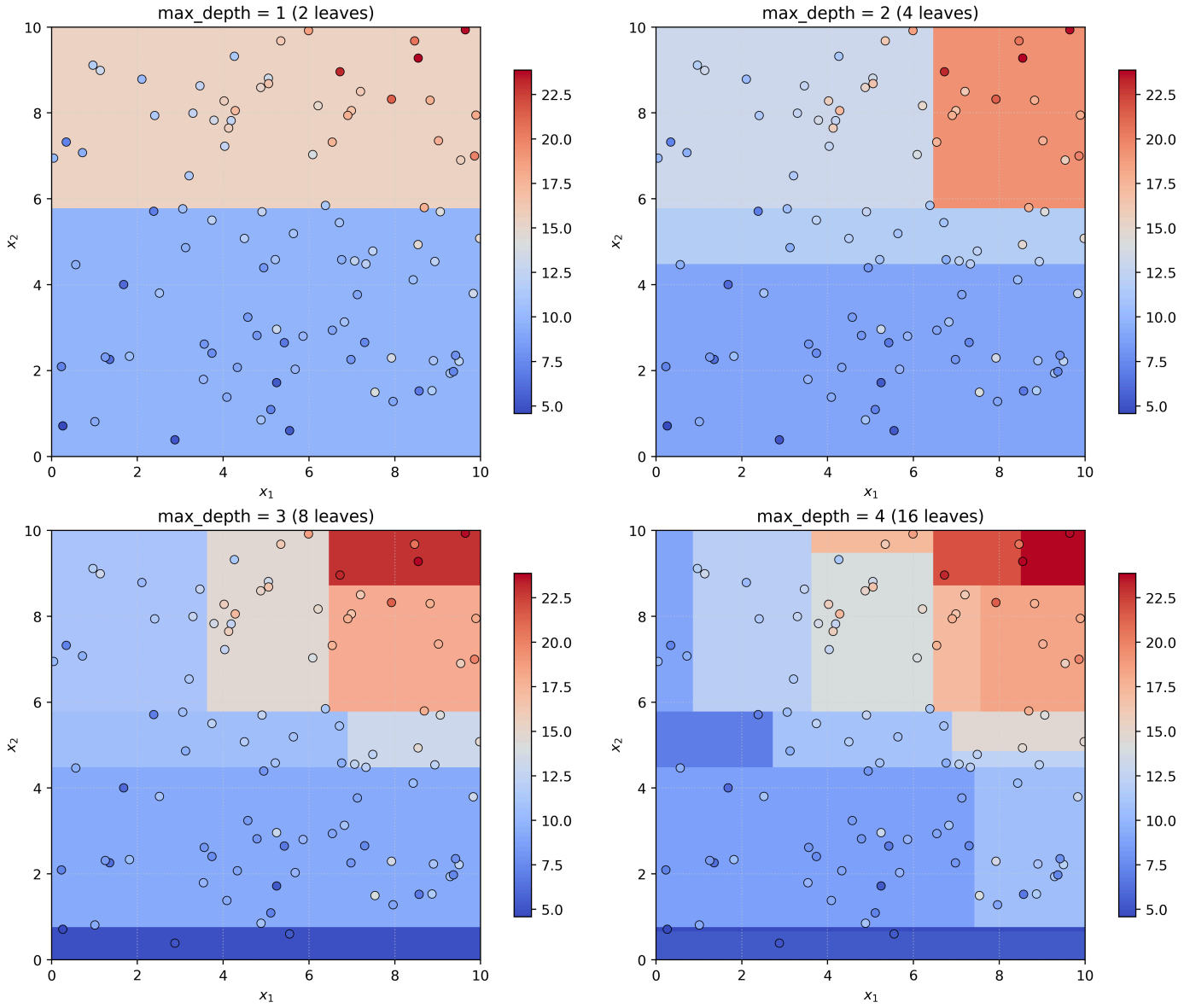


Figure 6: Demonstration of recursive partitioning process

Example: Automobile Fuel Efficiency

```
auto_mpg = fetch_ucirepo("Auto MPG")
```

We load the Auto MPG dataset using the `fetch_ucirepo` function, which provides access to datasets from the UCI Machine Learning Repository .

```
X = auto_mpg.data.features
y = auto_mpg.data.targets["mpg"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
```

```
    y,
    test_size=0.2,
    random_state=42,
)
```

```
DecisionTreeRegressor().get_params()
```

```
{'ccp_alpha': 0.0,
 'criterion': 'squared_error',
 'max_depth': None,
 'max_features': None,
 'max_leaf_nodes': None,
 'min_impurity_decrease': 0.0,
 'min_samples_leaf': 1,
 'min_samples_split': 2,
 'min_weight_fraction_leaf': 0.0,
 'monotonic_cst': None,
 'random_state': None,
 'splitter': 'best'}
```

```
tree = DecisionTreeRegressor(max_depth=2, random_state=42)
```

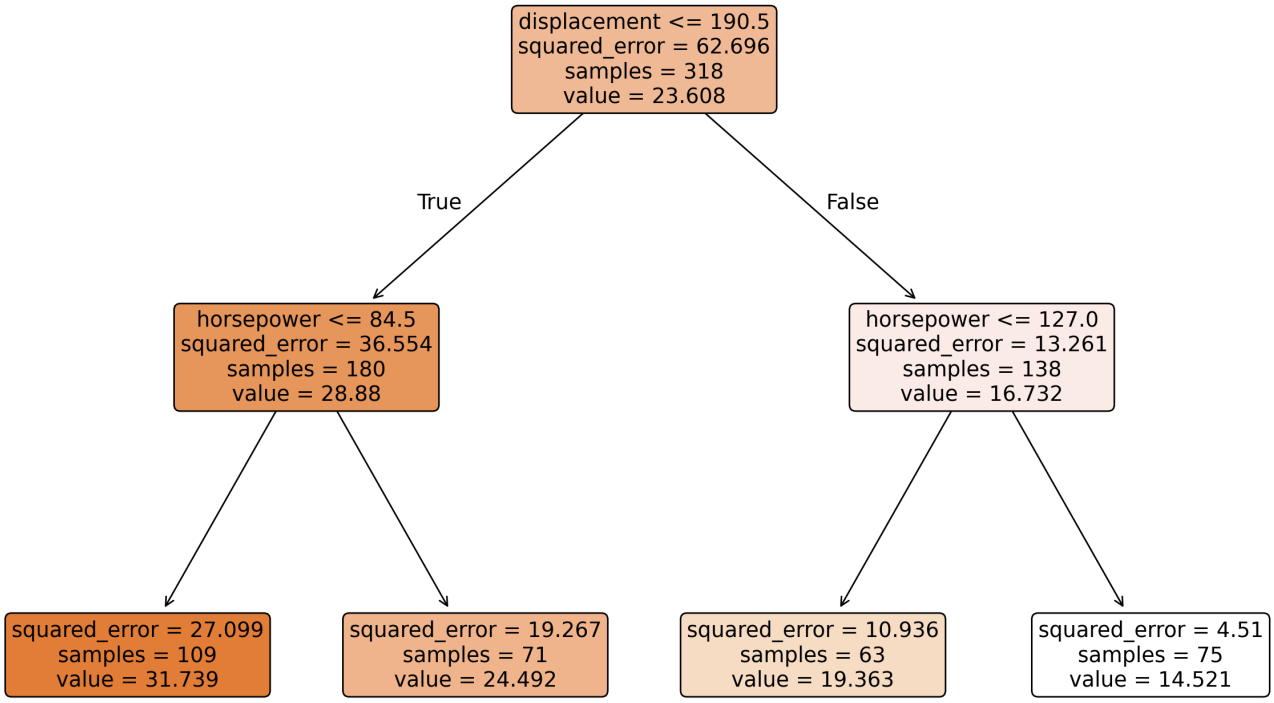
```
_ = tree.fit(X_train, y_train)
```

```
tree.predict(X_test)
```

```
array([31.7385, 31.7385, 19.3635, 14.5213, 14.5213, 24.4915, 24.4915,
       14.5213, 19.3635, 14.5213, 14.5213, 31.7385, 31.7385, 14.5213,
       31.7385, 14.5213, 24.4915, 24.4915, 14.5213, 31.7385, 31.7385,
       19.3635, 19.3635, 31.7385, 14.5213, 31.7385, 24.4915, 24.4915,
       19.3635, 14.5213, 24.4915, 31.7385, 19.3635, 24.4915, 31.7385,
       14.5213, 24.4915, 14.5213, 14.5213, 31.7385, 24.4915, 31.7385,
       24.4915, 14.5213, 24.4915, 24.4915, 31.7385, 24.4915, 24.4915,
       31.7385, 31.7385, 31.7385, 31.7385, 14.5213, 24.4915, 14.5213,
       14.5213, 24.4915, 24.4915, 19.3635, 14.5213, 31.7385, 31.7385,
       24.4915, 19.3635, 24.4915, 24.4915, 31.7385, 31.7385, 14.5213,
       31.7385, 14.5213, 14.5213, 19.3635, 24.4915, 19.3635, 19.3635,
       24.4915, 31.7385, 14.5213])
```

```
fig, ax = plt.subplots()
plot_tree(tree, feature_names=X_train.columns, filled=True, rounded=True)
```

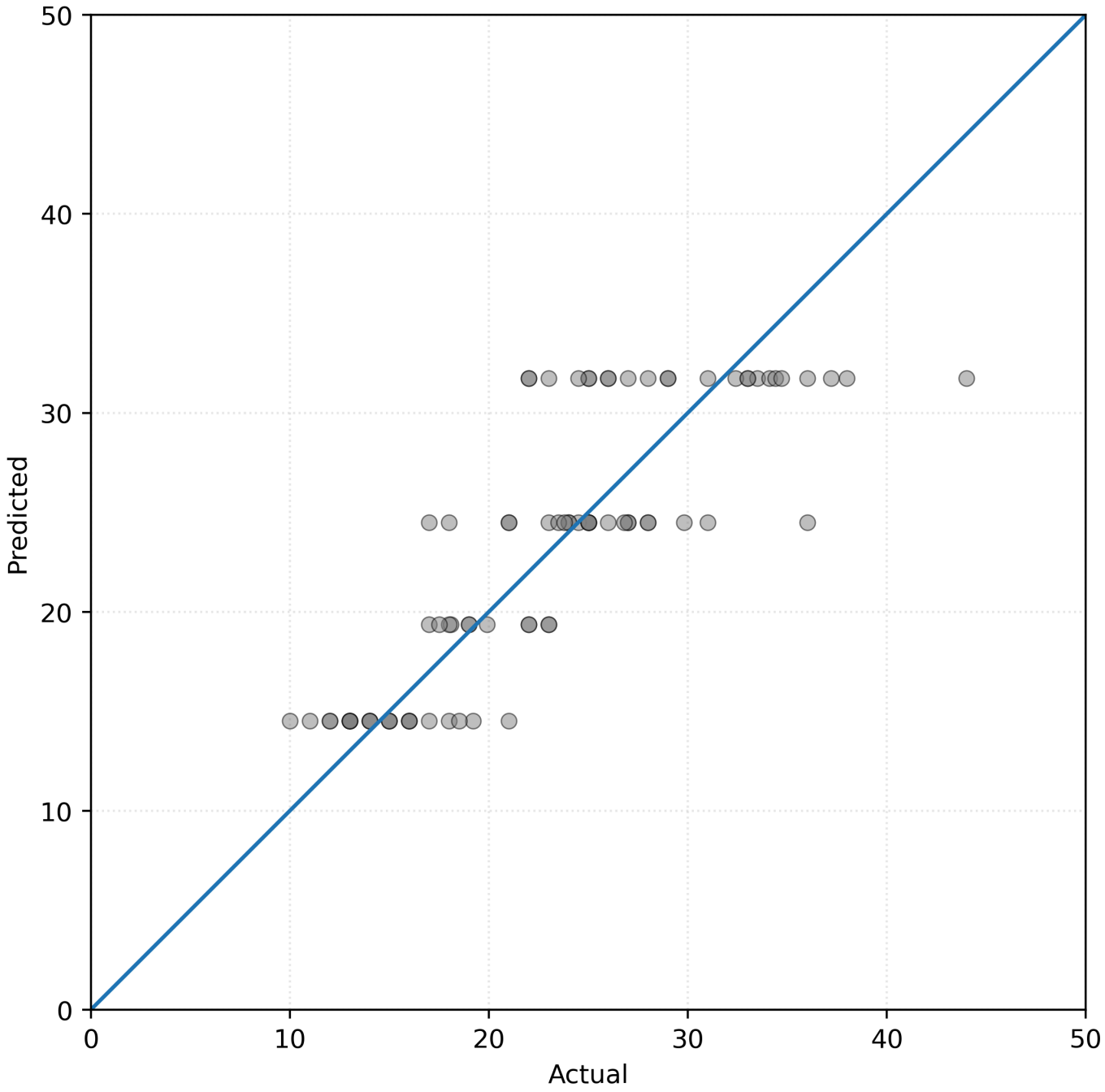
```
plt.show()
```




```
print(export_text(tree, feature_names=list(X_train.columns)))
```

```
|--- displacement <= 190.50
|   |--- horsepower <= 84.50
|   |   |--- value: [31.74]
|   |--- horsepower >  84.50
|   |   |--- value: [24.49]
|--- displacement >  190.50
|   |--- horsepower <= 127.00
|   |   |--- value: [19.36]
|   |--- horsepower >  127.00
|   |   |--- value: [14.52]
```

▶ Show Code for Plot


Predicted
Actual
0
10
20
30
40
50